

## 7주 차 보고서

( 2026년 2 월 08 일 ~ 2026년 2 월 14 일 )

작성자

배주환

### 이번 주 회의 내용

### 이번 주 한 일

```
// 옵션을 해석하고 처리할 주체?
// 소켓 코드 -> SOL_SOCKET
// IPv4 -> IPPROTO_IP
// TCP 프로토콜 -> IPPROTO_TCP

// SO_KEEPALIVE = 주기적으로 연결 상태 확인 여부 (TCP only)
// 상대방이 소리소문없이 연결 끊는다면?
// 주기적으로 TCP 프로토콜 연결 상태 확인 -> 끊어진 연결 감지

bool enable = true;
::setsockopt(serverSocket, SOL_SOCKET, SO_KEEPALIVE, (char*)&enable, sizeof(enable));

// SO_LINGER = 지연하다
// 송신 버퍼에 있는 데이터를 보낼 것인가? 날릴 것인가?

// onoff = 0이면 closesocket()이 바로 리턴. 아니면 linger초만큼 대기 (default 0)
// linger : 대기시간

LINGER linger;
linger.l_onoff = 1;
linger.l_linger = 5;
::setsockopt(serverSocket, SOL_SOCKET, SO_LINGER, (char*)&linger, sizeof(linger));

// Half-Close
// SO_SEND : send 막는다
// SO_RECEIVE : recv 막는다
// SO_BOTH : 둘다 막는다
//::shutdown(serverSocket, SO_SEND);

// 소켓 리소스 반환
// send -> closesocket
//::closesocket(serverSocket);

// SO_SNDBUF = 송신 버퍼 크기
// SO_RCVBUF = 수신 버퍼 크기

int32 sendBufferSize;
optionLen = sizeof(sendBufferSize);
::getsockopt(serverSocket, SOL_SOCKET, SO_SNDBUF, (char*)&sendBufferSize, &optionLen);
cout << "송신 버퍼 크기 : " << sendBufferSize << endl;

int32 recvBufferSize;
optionLen = sizeof(recvBufferSize);
::getsockopt(serverSocket, SOL_SOCKET, SO_RCVBUF, (char*)&recvBufferSize, &optionLen);
cout << "수신 버퍼 크기 : " << recvBufferSize << endl;
```

```

// SO_REUSEADDR
// IP주소 및 port 재사용
{
    bool enable = true;
    ::setsockopt(serverSocket, SOL_SOCKET, SO_REUSEADDR, (char*)&enable, sizeof(enable));
}

// IPPROTO_TCP
// TCP_NODELAY = Nagle 네이글 알고리즘 작동 여부
// 데이터가 충분히 크면 보내고, 그렇지 않으면 데이터가 충분히 쌓일 때까지 대기!
// 장점 : 작은 패킷이 불필요하게 많이 생성되는 일을 방지
// 단점 : 반응 시간 손해
{
    bool enable = true;
    ::setsockopt(serverSocket, IPPROTO_TCP, TCP_NODELAY, (char*)&enable, sizeof(enable));
}

```

이번주에는 소켓 옵션에 대해서 공부를 했다.

옵션을 해석하고 처리할 주체에는 SOL\_SOCKET, IPPROTO\_IP, IPPROTO\_TCP가 있다.

SO\_KEEPALIVE는 상대방이 소리소문없이 연결을 끊었는지 주기적으로 확인한다.

SO\_REUSEADDR은 서버 종료 후 바로 재시작할 때, 이미 사용중인 ip를 즉시 재사용할 수 있게 한다.

SO\_LINGER는 closesocket 호출 시, 송신 버퍼에 남은 데이터를 마저 보낼지 아니면 바로 날릴지 결정한다.

Half-Close(shutdown)에는 SD\_SEND(데이터 보내기만 중단), SD\_RECEIVE(데이터 받기만 중단), SD\_BOTH(둘 다 중단)이 있다.

SO\_SNDBUF/SO\_RCVBUF는 송수신 버퍼의 크기를 직접 확인하거나 변경할 때 사용한다.

TCP\_NODELAY는 Nagle알고리즘 작동여부를 결정한다. on이면 작은 데이터를 모아서 한번에 보내고, off면 모으지 않고 즉시 보낸다.

```

u_long on = 1;
if (::ioctlsocket(clientSocket, FIONBIO, &on) == INVALID_SOCKET)
    return 0;

```

논블로킹 소켓도 사용해보았다.

논블로킹 소켓은 예외처리를 모든 코드에 다 집어넣어야 한다는 점에서 논블로킹 소켓만으로는 잘 사용하지 않는다.

```
// Select 모델 = (select 함수가 핵심)
// 소켓 함수 호출이 성공할 시점을 미리 알 수 있다
// 문제 상황)
// 수신버퍼에 데이터가 없는데, read한다거나
// 송신버퍼가 꽉 찼는데, write한다던가
// - 블로킹 소켓 : 조건이 만족되지 않아서 블로킹되는 상황 예방
// - 논블로킹 소켓 : 조건이 만족되지 않아서 불필요하게 반복 체크하는 상황을 예방
// socket set
// 1) 읽기[ ] 쓰기[ ] 예외(OBB)[ ] 관찰 대상 등록
// 2) select(readSet, writeSet, exceptSet); -> 관찰 시작
// 3) 적어도 하나의 소켓이 준비되면 리턴 -> 낙오자는 알아서 제거됨
// 4) 남은 소켓 체크해서 진행

// fd_set set;
// FD_ZERO : 비운다
// ex) FD_ZERO(set);
// FD_SET : 소켓 s를 넣는다
// ex) FD_SET(s, &set);
// FD_CLR : 소켓 s를 제거
// ex) FD_CLR(s, &set);
// FD_ISSET : 소켓 s가 set에 들어있으면 0이 아닌 값을 리턴한다
```

```

vector<Session> sessions;
sessions.reserve(100);

fd_set reads;
fd_set writes;

while (true)
{
    // 소켓 셋 초기화
    FD_ZERO(&reads);
    FD_ZERO(&writes);

    // ListenSocket 등록
    FD_SET(listenSocket, &reads);

    for (Session& s : sessions)
    {
        if (s.recvBytes <= s.sendBytes)
            FD_SET(s.socket, &reads);
        else
            FD_SET(s.socket, &writes);
    }

    // [옵션] 마지막 timeout 인자 설정 가능
    int32 retVal = ::select(0, &reads, &writes, nullptr, nullptr);
    if (retVal == SOCKET_ERROR)
        break;

    // Listener 소켓 체크
    if (FD_ISSET(listenSocket, &reads))
    {
        SOCKADDR_IN clientAddr;
        int32 addrLen = sizeof(clientAddr);
        SOCKET clientSocket = ::accept(listenSocket, (SOCKADDR*)&clientAddr, &addrLen);
        if (clientSocket != INVALID_SOCKET)
        {
            cout << "Client Connected" << endl;
            sessions.push_back(Session{ clientSocket });
        }
    }
}

```

```

// 나머지 소켓 체크
for (Session& s : sessions)
{
    if (FD_ISSET(s.socket, &reads))
    {
        int32 recvLen = ::recv(s.socket, s.recvBuffer, BUFSIZE, 0);
        if (recvLen <= 0)
        {
            // TODO : sessions 제거
            continue;
        }

        s.recvBytes = recvLen;
    }

    // Write
    if (FD_ISSET(s.socket, &writes))
    {
        // 블로킹 모드 -> 모든 데이터 다 보냄
        // 논블로킹 모드 -> 일부만 보낼 수가 있음 (상대방 수신 버퍼 상황에 따라)
        int32 sendLen = ::send(s.socket, &s.recvBuffer[s.sendBytes], s.recvBytes - s.sendBytes, 0);
        if (sendLen == SOCKET_ERROR)
        {
            // TODO : sessions 제거
            continue;
        }

        s.sendBytes += sendLen;
        if (s.recvBytes == s.sendBytes)
        {
            s.recvBytes = 0;
            s.sendBytes = 0;
        }
    }
}

```

select 모델을 공부했다. select()함수는 여러 개의 소켓을 관찰하다가 어떤 소켓에서 사건이 발생하면 알려주는 역할을 한다. 코드의 동작 흐름은 이렇다. Fd\_set을 FD\_ZERO로 비우고 다시 채운다. Select 호출해서 관찰을 시작한다. 리스너 소켓을 체크한다. 클라이언트 소켓을 체크한다.

```

vector<WSAEVENT> wsaEvents;
vector<Session> sessions;
sessions.reserve(100);

WSAEVENT listenEvent = ::WSACreateEvent();
wsaEvents.push_back(listenEvent);
sessions.push_back(Session{ listenSocket });
if (::WSAEventSelect(listenSocket, listenEvent, FD_ACCEPT | FD_CLOSE) == SOCKET_ERROR)
    return 0;

while (true)
{
    int32 index = ::WSAWaitForMultipleEvents(wsaEvents.size(), &wsaEvents[0], FALSE, WSA_INFINITE, FALSE);
    if (index == WSA_WAIT_FAILED)
        continue;

    index -= WSA_WAIT_EVENT_0;

    // ::WSAResetEvent(wsaEvents[index]);

    WSANETWORKEVENTS networkEvents;
    if (::WSAEnumNetworkEvents(sessions[index].socket, wsaEvents[index], &networkEvents) == SOCKET_ERROR)
        continue;

    // Listener 소켓 체크
    if (networkEvents.lNetworkEvents & FD_ACCEPT)
    {
        // Error-Check
        if (networkEvents.iErrorCode[FD_ACCEPT_BIT] != 0)
            continue;

        SOCKADDR_IN clientAddr;
        int32 addrLen = sizeof(clientAddr);

        SOCKET clientSocket = ::accept(listenSocket, (SOCKADDR*)&clientAddr, &addrLen);
        if (clientSocket != INVALID_SOCKET)
        {
            cout << "Client Connected" << endl;

            WSAEVENT clientEvent = ::WSACreateEvent();
            wsaEvents.push_back(clientEvent);
            sessions.push_back(Session{ clientSocket });
            if (::WSAEventSelect(clientSocket, clientEvent, FD_READ | FD_WRITE | FD_CLOSE) == SOCKET_ERROR)
                return 0;
        }
    }
}

```

```

// Client Session 소켓 체크
if (networkEvents.INetworkEvents & FD_READ || networkEvents.INetworkEvents & FD_WRITE)
{
    // Error-Check
    if ((networkEvents.INetworkEvents & FD_READ) && networkEvents.iErrorCode[FD_READ_BIT] != 0)
        continue;

    // Error-Check
    if ((networkEvents.INetworkEvents & FD_WRITE) && networkEvents.iErrorCode[FD_WRITE_BIT] != 0)
        continue;

    Session& s = sessions[index];

    // Read
    if (s.recvBytes == 0)
    {
        int32 recvLen = ::recv(s.socket, s.recvBuffer, BUFSIZE, 0);
        if (recvLen == SOCKET_ERROR && ::WSAGetLastError() != WSAEWOULDBLOCK)
        {
            // TODO : Remove Session
            continue;
        }

        s.recvBytes = recvLen;
        cout << "Recv Data - " << recvLen << endl;
    }

    // Write
    if (s.recvBytes > s.sendBytes)
    {
        int32 sendLen = ::send(s.socket, &s.recvBuffer[s.sendBytes], s.recvBytes - s.sendBytes, 0);

        if (sendLen == SOCKET_ERROR && ::WSAGetLastError() != WSAEWOULDBLOCK)
        {
            // TODO : Remove Session
            continue;
        }

        s.sendBytes += sendLen;
        if (s.recvBytes == s.sendBytes)
        {
            s.recvBytes = 0;
            s.sendBytes = 0;
        }

        cout << "Send Data - " << sendLen << endl;
    }
}

```

wsaEventSelect 모델은 select모델의 단점을 보완하기 위해 이벤트 객체를 활용하는 방식이다. Select가 매 번 루프를 돌며 모든 소켓을 검사했다면, 이 모델은 사건이 발생한 소켓에 대응하는 이벤트 객체가 신호를 줄 때까지 기다리는 방식이다.

소켓과 이벤트 객체를 1대1로 대응시켜 관리하는데 소켓에 네트워크 이벤트가 발생하면, 연결된 이벤트 객체가 signaled 상태로 바뀐다. WSAWaitForMultipleEvents 함수를 사용하여 여러 이벤트 중 하나라도 신호가 올때까지 효율적으로 대기한다.

코드의 동작 흐름은 이러하다. 소켓마다 WSACreateEvent() 로 이벤트 객체를 생성한다. WSAEventSelect()를 통해 어떤 네트워크 사건을 감시할지 소켓에 등록한다. wsaEvents 배열에 담긴 이벤트 객체들을 관찰하며 기다린다. 이벤트가 발생하면 해당 이벤트의 인덱스를 반환받아 어떤 소켓에서 일이 터졌는지 즉시 알 수 있다. WSAEnumNetworkEvents를 호출하여 실제 발생한 구체적인 이벤트가 무엇인지를 networkEvents 구조체에 받아온다. 이 함수를 호출하면 자동으로 이벤트 객체는 non-signaled 상태로 리셋된다.

```

// Overlapped IO (비동기 + 논블로킹)
// - Overlapped 함수를 건다 (WSARecv, WSARecv)
// - Overlapped 함수가 성공했는지 확인 후
// -> 성공했으면 결과 얻어서 처리
// -> 실패했으면 사유를 확인

// 1) 비동기 입출력 소켓
// 2) WSABUF 배열의 시작 주소 + 개수 // Scatter-Gather
// 3) 보내고/받은 바이트 수
// 4) 상세 옵션인데 0
// 5) WSAOVERLAPPED 구조체 주소값
// 6) 입출력이 완료되면 OS가 호출할 콜백 함수
// WSARecv
// WSARecv

// Overlapped 모델 (이벤트 기반)
// - 비동기 입출력 지원하는 소켓 생성 + 통지 받기 위한 이벤트 객체 생성
// - 비동기 입출력 함수 호출 (1에서 만든 이벤트 객체를 같이 넘겨줌)
// - 비동기 작업이 바로 완료되지 않으면, WSA_IO_PENDING 오류 코드
// 운영체제는 이벤트 객체를 signaled 상태로 만들어서 완료 상태 알려줌
// - WSAPoll 함수 호출해서 이벤트 객체의 signal 판별
// - WSAGetOverlappedResult 호출해서 비동기 입출력 결과 확인 및 데이터 처리

// 1) 비동기 소켓
// 2) 넘겨준 overlapped 구조체
// 3) 전송된 바이트 수
// 4) 비동기 입출력 작업이 끝날때까지 대기할지?
// false
// 5) 비동기 입출력 작업 관련 부가 정보. 거의 사용 안함
// WSAGetOverlappedResult

```



```

while (true)
{
    SOCKADDR_IN clientAddr;
    int32 addrLen = sizeof(clientAddr);

    SOCKET clientSocket;
    while (true)
    {
        clientSocket = ::accept(listenSocket, (SOCKADDR*)&clientAddr, &addrLen);
        if (clientSocket != INVALID_SOCKET)
            break;

        if (::WSAGetLastError() == WSAEWOULDBLOCK)
            continue;

        // 문제 있는 상황
        return 0;
    }

    Session session = Session{ clientSocket };
    WSAEVENT wsaEvent = ::WSACreateEvent();
    session.overlapped.hEvent = wsaEvent;

    cout << "Client Connected!" << endl;

    while (true)
    {
        WSABUF wsaBuf;
        wsaBuf.buf = session.recvBuffer;
        wsaBuf.len = BUFSIZE;

        DWORD recvLen = 0;
        DWORD flags = 0;
        if (::WSARecv(clientSocket, &wsaBuf, 1, &recvLen, &flags, &session.overlapped, nullptr) == SOCKET_ERROR)
        {
            if (::WSAGetLastError() == WSA_IO_PENDING)
            {
                // Pending
                ::WSAWaitForMultipleEvents(1, &wsaEvent, TRUE, WSA_INFINITE, FALSE);
                ::WSAGetOverlappedResult(session.socket, &session.overlapped, &recvLen, FALSE, &flags);
            }
            else
            {
                // TODO : 문제 있는 상황
                break;
            }
        }

        cout << "Data Recv Len = " << recvLen << endl;
    }
}

```

Overlapped 모델을 이벤트 기반과 callback 방식으로 만들 수 있는데 이벤트 기반으로 먼저 만들어보았다. 비동기 + 논블로킹 구조로 데이터가 올 때까지 기다리지 않고, 예약만 해둔 뒤 나중에 확인한다는 점이 특징이다. 예약을 확인하는 방법은 이벤트 기반으로 작업이 완료되면 WSAEVENT가 signaled 상태로 변하여 이를 통해 작업 완료를 감지한다.

```

// Overlapped 모델 (Completion Routine 콜백 기반)
// - 비동기 입출력 지원하는 소켓 생성
// - 비동기 입출력 함수 호출 (완료 루틴의 시작 주소를 넘겨준다)
// - 비동기 작업이 바로 완료되지 않으면, WSA_IO_PENDING 오류 코드
// - 비동기 입출력 함수 호출한 쓰레드를 -> Alertable Wait 상태로 만든다
// ex) WaitForSingleObjectEx, WaitForMultipleObjectsEx, SleepEx, WSAWaitForMultipleEvents
// - 비동기 IO 완료되면, 운영체제는 완료 루틴 호출
// - 완료 루틴 호출이 모두 끝나면, 쓰레드는 Alertable Wait 상태에서 빠져나온다

// 1) 오류 발생시 0 아닌 값
// 2) 전송 바이트 수
// 3) 비동기 입출력 함수 호출 시 넘겨준 WSAOVERLAPPED 구조체의 주소값
// 4) 0
// void CompletionRoutine()

// Select 모델
// - 장점) 윈도우/리눅스 공통.
// - 단점) 성능 최하 (매번 등록 비용), 64개 제한
// WSAEventSelect 모델
// - 장점) 비교적 뛰어난 성능
// - 단점) 64개 제한
// Overlapped (이벤트 기반)
// - 장점) 성능
// - 단점) 64개 제한
// Overlapped (콜백 기반)
// - 장점) 성능
// - 단점) 모든 비동기 소켓 함수에서 사용 가능하진 않음 (accept), 빈번한 Alertable Wait로 인한 성능 저하
// IOCP

// Reactor Pattern (~뒤늦게, 논블로킹 소켓, 소켓 상태 확인 후 -> 뒤늦게 recv send 호출)
// Proactor Pattern (~미리, Overlapped WSA~)

struct Session
{
    SOCKET socket = INVALID_SOCKET;
    char recvBuffer[BUFSIZE] = {};
    int32 recvBytes = 0;
    WSAOVERLAPPED overlapped = {};
};

void CALLBACK RecvCallback(DWORD error, DWORD recvLen, LPWSAOVERLAPPED overlapped, DWORD flags)
{
    cout << "Data Recv Len Callback = " << recvLen << endl;
    // TODO : 에코 서버를 만든다면 WSA_send()
}

```

```

while (true)
{
    SOCKADDR_IN clientAddr;
    int32 addrLen = sizeof(clientAddr);

    SOCKET clientSocket;
    while (true) { ... }

    Session session = Session( clientSocket );
    //WSAEVENT wsaEvent = ::WSACreateEvent();

    cout << "Client Connected!" << endl;

    while (true)
    {
        WSABUF wsaBuf;
        wsaBuf.buf = session.recvBuffer;
        wsaBuf.len = BUFSIZE;

        DWORD recvLen = 0;
        DWORD flags = 0;
        if (::WSARecv(clientSocket, &wsaBuf, 1, &recvLen, &flags, &session.overlapped, RecvCallback) == SOCKET_ERROR)
        {
            if (::WSAGetLastError() == WSA_IO_PENDING)
            {
                // Pending
                // Alertable Wait
                ::SleepEx(INFINITE, TRUE);
                //::WSAWaitForMultipleEvents(1, &wsaEvent, TRUE, WSA_INFINITE, TRUE);
            }
            else
            {
                // TODO : 문제 있는 상황
                break;
            }
        }
        else
        {
            cout << "Data Recv Len = " << recvLen << endl;
        }
    }
}

```

Callback방식도 만들어보았다. 작업이 끝나면 이 callback함수를 실행해달라고 OS에 맡기는 방식이 특징이다. SleepEx에서 마지막인자를 TRUE로 설정하여 OS가 콜백 함수를 실행할 수 있도록 스레드를 잠시 비워주는 특수한 대기 상태로 들어간다.

## 다음 주 할 일

IOCP모델에 대해 공부를 하고, 네트워크 라이브러리를 제작할 예정이다.

## 특이사항

